

Sprawozdanie z projektu: Implementacja indeksowo-sekwencyjnej organizacji pliku

1. Przyjęta metoda indeksowania

Do realizacji zadania przyjęto organizację indeksowo-sekwencyjną (ISAM - Indexed Sequential Access Method). Struktura ta pozwala na szybki dostęp do danych poprzez rzadki indeks (sparse index) oraz obsługę rekordów nadmiarowych (overflow) powstających w wyniku wstawiania danych do pełnych stron.

System składa się z trzech plików fizycznych:

1. Plik główny (data.txt): Przechowuje posortowane rekordy w stronach (blokach) o stałym rozmiarze $b=4$.
2. Plik indeksu (index.txt): Przechowuje pary (Klucz, Numer_Strony), wskazujące na najmniejszy lub największy klucz na danej stronie w pliku głównym.
3. Plik nadmiarowy (overflow.txt): Przechowuje rekordy, które nie zmieściły się w pliku głównym, tworząc listy jednokierunkowe.
4. Opis implementacji

Program został zrealizowany w języku Python w podejściu obiektowym. Główną logiką steruje klasa Manager.

Struktura Rekordu i Strony:

- Rekord: Składa się z klucza (8 bajtów), 4 liczb zmiennoprzecinkowych (dane), wskaźnika na obszar nadmiarowy (overflow) oraz znacznika usunięcia (deleted).
- Strona (Blok): Przyjęto współczynnik blokowania $b=4$. Strona jest najmniejszą jednostką odczytu/zapisu.

Zarządzanie Pamięcią (Buforowanie): W celu zminimalizowania operacji dyskowych zastosowano prosty mechanizm buforowania:

- W pamięci operacyjnej (RAM) przechowywana jest zawsze tylko jedna, aktualnie przetwarzana strona dla każdego z trzech plików (strona danych, strona indeksu, strona nadmiaru).
- Zastosowano mechanizm "dirty bit" (flaga dirty). Zapis strony na dysk następuje tylko wtedy, gdy jej zawartość została zmodyfikowana, a program musi załadować inną stronę lub zakończyć działanie.

Obsługa Nadmiarów (Overflow): Gdy strona w pliku głównym jest pełna, nowy rekord trafia do pliku overflow.txt.

- Rekordy tworzą listę jednokierunkową.
- Wskaźnik (pointer) to liczba całkowita obliczana jako: $\text{numer_strony} * b + \text{offset_na_stronie}$.
- Rekord w pliku głównym wskazuje na pierwszy element w łańcuchu nadmiaru, a kolejne rekordy w overflow wskazują na następne. Łańcuch jest utrzymywany w porządku rosnącym kluczy, co przyspiesza wyszukiwanie.

Reorganizacja: Zaimplementowano reorganizację automatyczną oraz na żądanie.

- Warunek automatyczny: Reorganizacja następuje, gdy liczba stron w obszarze nadmiarowym przekroczy ustalony próg (zależny od parametru `reorg_threshold` i wielkości pliku głównego).
- Przebieg: Program tworzy nowe pliki, przepisując sekwencyjnie wszystkie aktywne (nieusunięte) rekordy z pliku głównego i nadmiarowego, scalając je i sortując.
- Współczynnik Alpha: Podczas reorganizacji strony w nowym pliku głównym są wypełniane tylko w stopniu określonym przez parametr Alpha (np. dla $\text{Alpha}=0.5$ i $b=4$, na stronie zapisywane są 2 rekordy, a 2 miejsca pozostają puste na przyszłe wstawienia).

Usuwanie i Aktualizacja:

- Usuwanie: Realizowane logicznie poprzez ustawienie flagi deleted = 1. Fizyczne usunięcie rekordu i odzyskanie miejsca następuje dopiero podczas reorganizacji.
- Aktualizacja: Zmienia zawartość rekordu (wektor danych). Jeśli klucz nie ulega zmianie, pozycja rekordu pozostaje ta sama.

2. Specyfikacja formatu pliku testowego

Program obsługuje wczytywanie komend z pliku tekstowego. Każda linia zawiera jedną instrukcję. Format komend:

- A [klucz] : Dodanie rekordu (dane losowane) (np. A 150).
- D [klucz] : Usunięcie rekordu (np. D 150).
- U [klucz] [v1] [v2] [v3] [v4] : Aktualizacja danych rekordu (np. U 150 1.1 2.2 3.3 4.4).
- S [klucz] : Wyszukanie rekordu (np. S 150).

3. Sposób prezentacji wyników

Program działa w trybie interaktywnym (menu) i po każdej operacji raportuje:

1. Licznik operacji dyskowych: Oddzielnie dla odczytów (Reads) i zapisów (Writes). Licznik jest resetowany przed każdą nową komendą użytkownika.
2. Struktura fizyczna (Debug Dump): Wyświetla zawartość stron dyskowych (zarówno głównego pliku, jak i overflow) w formacie czytelnym dla człowieka, pokazując klucze, dane oraz wskaźniki.
3. Widok logiczny: Wyświetla rekordy posortowane (złożenie pliku głównego i łańcuchów nadmiaru).
4. Opis przeprowadzonych eksperymentów

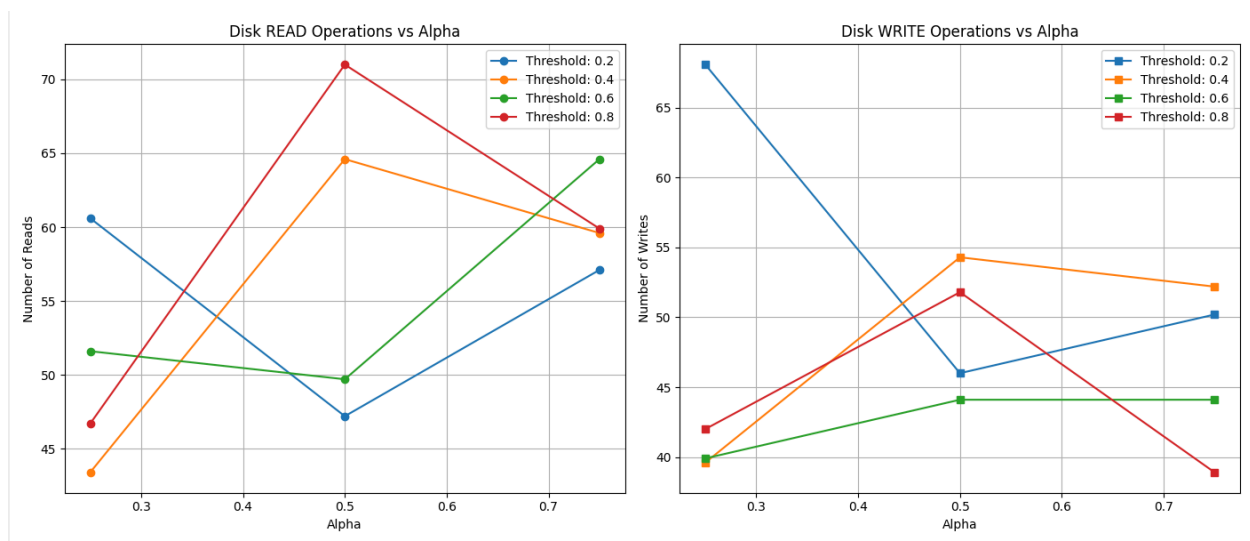
Celem eksperymentu było zbadanie wpływu współczynnika wypełnienia strony

(Alpha) oraz progu reorganizacji (threshold) na wydajność operacji dyskowych.

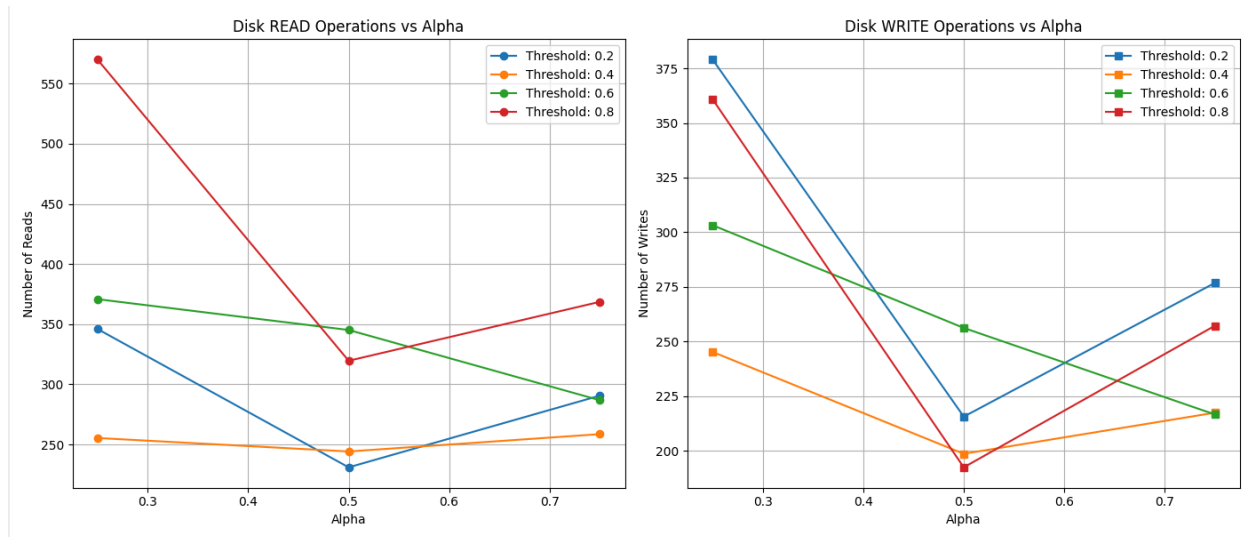
Konfiguracja eksperymentu:

- Dane: Zestawy losowych rekordów.
- 2 Warianty 1: 200 rekordów (mały zbiór), 500 rekordów (średni zbiór)
- Zmienne niezależne: a) Alpha: 0.25, 0.5, 0.75. b) Próg reorganizacji: 0.2, 0.4, 0.6, 0.8 (ułamek wielkości pliku głównego, po którym następuje przebudowa).
- Zmienne zależne: Średnia liczba odczytów i zapisów dyskowych.

Wykres dla 200 rekordów:



Wykres dla 500 rekordów:



Poniżej przedstawiono analizę na podstawie wygenerowanych wykresów.

A. Analiza operacji ZAPISU (Writes):

- Niski próg reorganizacji (np. 0.2 - linia niebieska): Powoduje bardzo dużą liczbę zapisów, zwłaszcza przy małym Alpha. Dzieje się tak, ponieważ system "panikuje" i zbyt często przebudowuje cały plik, co jest operacją bardzo kosztowną (przepisanie całej bazy).
- Wysoki próg reorganizacji (np. 0.8 - linia czerwona): Liczba zapisów jest stabilna i niska. System rzadziej wykonuje kosztowną reorganizację, po prostu dopisując rekordy do pliku overflow.
- Wniosek: Częsta reorganizacja drastycznie zwiększa koszt zapisu.

B. Analiza operacji ODCZYTU (Reads):

- Wpływ Alpha: Przy wysokim Alpha (0.75) i wysokim progu reorganizacji (0.8 - linia czerwona na wykresie 200 rekordów), liczba odczytów gwałtownie rośnie.
 - Przyczyna: Wysokie Alpha oznacza, że po reorganizacji strony są prawie pełne (tylko 1 wolne miejsce przy $b=4$). Nowe rekordy natychmiast trafiają do overflow.
 - Skutek: Tworzą się długie łańcuchy nadmiaru. Aby znaleźć rekord, trzeba przeczytać stronę główną, a potem skakać po stronach

nadmiaru, co generuje wiele odczytów.

- Niski próg reorganizacji (0.2): Utrzymuje liczbę odczytów na niskim poziomie (płaska niebieska linia), ponieważ struktura jest ciągle "świeża" i uporządkowana, a łańcuchy nadmiaru są krótkie.

C. Porównanie skali (200 vs 500 rekordów):

- Dla 500 rekordów (Wykres 2) obserwujemy te same trendy, ale wartości bezwzględne są znacznie wyższe (np. odczyty sięgają 550 przy złej konfiguracji, w porównaniu do 75 przy 200 rekordach). Wskazuje to na nieliniowy wzrost kosztów przeszukiwania przy degradacji struktury pliku (długie łańcuchy overflow).

4. Wnioski końcowe

1. Kompromis (Trade-off): Istnieje wyraźny konflikt między wydajnością zapisu a odczytu.
 - a. Częsta reorganizacja (niski próg) = Idealne odczyty, ale tragiczne zapisy.
 - b. Rzadka reorganizacja (wysoki próg) = Tanie zapisy, ale kosztowne odczyty (długie łańcuchy).
2. Optymalne parametry:
 - a. Dla zaimplementowanego systemu (ISAM, $b=4$) optymalnym punktem wydaje się być Alpha ok. 0.5 oraz Próg reorganizacji ok. 0.4 - 0.6.
 - b. Pozwala to zachować równowagę: zostawiamy 50% miejsca na przyszłe wstawienia (opóźniając użycie overflow), a reorganizację robimy umiarkowanie często, by nie "zajechać" dysku zapisami, ale też nie dopuścić do powstania tasemcowych list nadmiaru.
3. Alpha: Ustawienie Alpha zbyt blisko 1.0 (np. 0.8) jest nieefektywne w systemach z dużą liczbą wstawień, gdyż korzyść z gęstego upakowania danych jest natychmiast niwelowana przez konieczność obsługi nadmiarów.